

IDONTKNOWITSNAME

Anonymous Author(s)

Abstract

LLM agent frameworks increasingly expose model invocations through structured programming abstractions, with framework runtimes lowering program-level constructs into concrete prompts. We observe that these programmatically generated prompts are inherently heterogeneous: their components exhibit different sharing scopes, lifetimes, and evolution patterns across invocations. However, existing serving systems treat prompts as opaque text, losing this structure and missing opportunities for more effective KV-cache reuse and management.

We present IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME’s compiler pass performs static analysis and compiler-level optimizations to expose binding lifetimes, expose application-level structure and proactively re-layout prompts to improve cache reuse. The serving runtime exploits the heterogeneity information to optimize KV-cache behavior across requests by prefetching, protection and eviction, according to the predicted future utility of cache entries. We implemented IDONTKNOWITSNAME on SGLang and evaluated it against multiple established serving systems. Our results shows that IDONTKNOWITSNAME.....

1 Introduction

Agent frameworks increasingly expose LLM calls as programming abstractions rather than manual prompt construction. ByLLM [3] introduces a language-level abstraction that delegates function execution to an LLM, with the runtime using semantic information from the code like function signatures and parameter names to construct model requests. NVIDIA Object-Oriented Agents (NOOA) [6] similarly embeds LLM-driven behavior into native Python objects and translates program-level declarations into prompts and model interactions. DSPy [10] represents LM invocations through declarative signatures and modules, while the framework automatically generates a concrete prompt. Across these systems, developers specify LLM behavior using structured programming constructs, while framework runtimes translate those constructs into concrete prompts and model requests.

This trend extends beyond hand-written agent code. SIGIL [4] compiles natural-language skill documents into programs with explicit control flow and typed LLM calls, which the runtime lowers into prompts. Increasingly, requests arriving at inference engines encode structure inherited from the programs that generate them. Existing serving systems, however, make limited use of this structure. Engines such as SGLang [19] exploit exact token-prefix matches for KV-cache reuse without recognizing the program components

within a prompt. Recent agent-serving systems, including PBKV [18], CacheScout [17], KVFlow [13], and Pythia [16], exploit workflow information to anticipate future invocations and guide cache management, but largely overlook the information encoded in the prompt itself. As a result, these systems fail to fully leverage the structure and semantics of programmatically generated prompts, leading to suboptimal cache management and performance.

We argue that these programmatically generated LLM requests are inherently heterogeneous. Components within a generated prompt can have different sharing scopes and lifetimes. For example, some components may remain fixed across invocations, some are shared across call sites, and some can grow or be replaced as execution progresses. As the example in Figure 1 shows, the four bindings of `assess_evidence` have distinct lifetimes and evolution patterns. A developer may naturally place `search_results` and `question` before evidences in the function signature, and the runtime preserves this ordering when lowering the invocation into a prompt. However, existing serving frameworks treat the requests as opaque text. Once requests reach the serving system, the original program structure is no longer visible, creating a gap between the application-level runtime and the serving layer. Because the serving engine blindly consumes the prompt in this order, any change to either binding prevents prefix-cache reuse of the unchanged entries in evidences. Additionally, because the serving layer holds no knowledge about the binding heterogeneity, it fails to identify which KV-cache entries are evictable, which leads to suboptimal management.

To close this gap, we propose IDONTKNOWITSNAME, a serving layer that exploits prompt heterogeneity to achieve more efficient KV-cache management. IDONTKNOWITSNAME recovers each request’s call site and binding boundaries by lifting the prompt text against the framework’s templates, learns heterogeneous binding evolution patterns and application-level structure from traces, and then conducts progressive KV-cache management. IDONTKNOWITSNAME exploits such heterogeneity at two levels: within a request, it reorders bindings by lifetime so that stable content forms a shareable prefix; across requests, a lifecycle-aware planner uses the learned control flow to prefetch prefixes before they are needed and evicts those that will not be reused. We implemented IDONTKNOWITSNAME on SGLang [19]. Our evaluation shows that IDONTKNOWITSNAME achieves up to $1.4\times$ – $1.7\times$ lower mean JCT and $1.7\times$ – $1.75\times$ lower mean TTFT than vanilla SGLang. IDONTKNOWITSNAME also outperforms multiple established serving systems.

In summary, we make the following contributions.

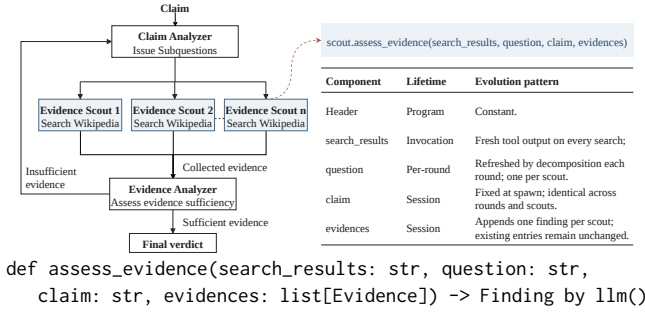


Figure 1. Heterogeneity within a HoVer-style [9] Fact Check Agent’s generated prompt for the summary-making function. `assess_evidence(search_results, question, claim, evidences)` evaluates the sub-question against the accumulated evidence and search results, then answers it.

- We identify the inherent heterogeneity in program-matically generated prompts and its implications for serving.
- We design and build `IDONTKNOWITSNAME`, a serving layer that exploits prompt heterogeneity to guide more efficient KV-cache scheduling.
- We conduct comprehensive evaluations of `IDONTKNOWITSNAME` against multiple established serving systems, demonstrating that `IDONTKNOWITSNAME` achieves up to $1.4\times-1.7\times$ lower mean JCT and $1.7\times-1.75\times$ lower mean TTFT than vanilla SGLang.

2 Background

2.1 From program constructs to prompts

Agent frameworks separate the declaration of an LLM operation from the construction of its input. In ByLLM, a developer supplies a function signature, types, and semantic descriptions, and the runtime constructs the prompt and parses the response [3]. DSPy expresses operations through signatures with named input and output fields, which adapters translate into model messages [10]. We use *call site* to denote an identifiable LLM operation in an application, and *binding* to denote a named input together with its value in a particular invocation. A *session* is one execution of a user-level task, potentially involving repeated calls to several sites and intervening tool executions.

For the template-based requests considered here, lowering combines framework instructions, call-site metadata, serialized bindings, and output-format instructions. The metadata can include a qualified function name, parameter types, and field descriptions. ByLLM-style rendering labels argument values by name; DSPy’s ChatAdapter uses explicit field delimiters [5]. Figure 2 illustrates these two forms. Their concrete message layouts differ, but both retain boundaries between named values and surrounding instructions. With the declaration, adapter configuration, and rendering rules fixed,

ByLLM-style binding layout (abridged)

```
Coder.implement(spec, plan, advice, attempts)
spec = <function specification>
plan = <implementation plan>
advice = <feedback for this attempt>
attempts = <accumulated attempt records>
```

DSPy ChatAdapter field layout (abridged)

```
[ [ ## spec ## ] ] <function specification>
[ [ ## plan ## ] ] <implementation plan>
[ [ ## advice ## ] ] <feedback for this attempt>
[ [ ## attempts ## ] ] <accumulated records>
```

Figure 2. Schematic rendering of the same coding inputs using named bindings and delimited fields. System instructions, schemas, and output instructions are omitted. The field values have different lifetimes despite occupying the same message.

repeated invocations can retain a common skeleton while their binding values change.

2.2 Prefix reuse and cache placement

An autoregressive transformer processes input tokens during *prefill*, producing per-layer key and value tensors, and then generates output tokens during *decode*. Keeping these tensors avoids recomputing earlier tokens during generation. PagedAttention, introduced in vLLM, manages KV memory in blocks to reduce fragmentation and support sharing [11]. SGLang’s RadixAttention indexes cached token sequences in a radix tree to reuse common prefixes across requests [19].

Under ordinary causal attention, a token’s cached state depends on its preceding context. Consequently, exact prefix caching requires matching tokens and compatible model and positional settings: identical text after a changed prefix cannot generally reuse its old KV state. A cache lookup can reuse the resident portion of the longest common token prefix; the remaining input requires prefill. This is a constraint on the serialized token sequence, independent of whether two later fields carry the same application-level value.

Cache availability imposes a second constraint. Hierarchical caching retains KV outside GPU memory and transfers it back when needed. CachedAttention combines hierarchical storage with overlapped loading and saving for multi-turn conversations [7]; Mooncake organizes distributed KV storage around disaggregated prefill and decode resources [14]. In the device–host setting used by `IDONTKNOWITSNAME`, a device hit avoids both transfer and recomputation, a host hit requires promotion, and a miss in both tiers requires recomputing the missing prefix. Chunked prefill can interleave this computation with decoding, as in Sarathi-Serve [1], but does not eliminate the work.

These costs affect both per-call time to first token (TTFT) and session job completion time (JCT). A session can revisit a call site after search, testing, or another LLM operation, during which competing requests may displace its cache.

Efficient reuse therefore depends on both the prefix exposed by prompt construction and the availability of that prefix when execution returns.

3 Motivation

3.1 Lifetime heterogeneity within one prompt

The coding example in Figure 2 illustrates why treating an entire prompt as one reuse unit loses useful information. Consider an iterative implementation task in which spec and plan remain fixed, advice is replaced after a failed test, and attempts accumulates earlier implementations and test results. These fields belong to the same invocation, yet their next values have different relationships to the current ones. The specification and plan can be copied, part of the attempt history remains known, and the next feedback is unavailable until testing completes. Framework instructions may persist even across sessions, whereas the specification is private to one task.

The fact-checking example in Figure 1 has the same structure: the claim persists across a verification session, evidence accumulates, and a new sub-question or search result can replace the corresponding input. Moreover, the same claim or evidence may be passed between call sites. This creates two distinct dimensions of heterogeneity: how a binding evolves over time, and where its value is shared. A field can be stable within one session without being a program-wide constant, and it can be shared across sites without remaining unchanged forever. These properties must be inferred from values and execution history; a parameter name or type alone does not establish them.

3.2 Declaration order hides reusable content

Frameworks organize fields to express an operation’s interface. That order need not expose the longest reusable token prefix. Suppose the coding request is rendered as

$$P_t = H||S||L||A_t||E_t||R,$$

where H contains fixed leading instructions and metadata, S is the specification, L is the plan, A_t is current advice, E_t is the attempt history, and R contains trailing instructions. Each field includes its label and delimiters. When advice changes, prefix matching stops within A_t even if most of E_t remains identical. Keeping the entire previous request in GPU memory cannot recover this lost reuse: the old history tensors were computed under the old advice.

Placing the attempt history before advice instead gives

$$P'_t = H||S||L||E_t||A_t||R.$$

If consecutive serialized histories share an initial segment, the reusable prefix can now extend through that segment. Appending a list element may replace a closing delimiter, so the reusable region is the actual common token prefix, not necessarily the complete previous field. This example

requires no prediction of the next advice. It exploits the part of the next request that is already known.

Cross-call-site sharing introduces another boundary. Two sites may receive identical leading bindings but have different headers. A site-specific header before those bindings prevents sharing across sites at that point. Moving shared bindings ahead of that header can expose further reuse when the system prompts, serialization, and preceding tokens also match. Identical binding values alone are insufficient. These observations motivate learning a consistent layout that considers both sharing scope and evolution, while retaining labels and field contents.

3.3 Reuse opportunities need timely residency

A longer common prefix is useful only if its KV state is available. Recency alone cannot distinguish an intermediate result that another call will soon consume from a fresh diagnostic that the next attempt will replace. Nor does a recent access imply continued usefulness after its session terminates. Conversely, a prefix can remain useful throughout a tool execution despite receiving no accesses during that interval. Continuum addresses retention across such pauses through cache time-to-live, while CacheScout learns execution transitions to guide retention and prefetching [12, 17].

Binding heterogeneity supplies information beyond the identity of a future call. Predicting that implement will run again does not exploit its full potential. Its specification and plan can be reconstructed from the live session, the history contributes an existing prefix, and new advice remains unresolved. Re-layout can place these reconstructible inputs before the first unknown field, increasing the amount of KV that can be prepared before arrival. Call-transition and timing observations could determine which candidate prefixes are likely to be needed and when promotion should begin.

4 Design

We designed IDONTKNOWITSNAME, a serving layer that exploits the heterogeneity of programmatically generated LLM requests to schedule the KV cache more efficiently, closing the gap between the runtime that generates prompts and the engine that serves them.

IDONTKNOWITSNAME employs two planes to comprehensively discover and exploit such heterogeneity. The discovery plane first lifts each incoming prompt back to its call-site signature and bindings, then combines this metadata with request history to learn how each binding evolves and the structure of the agentic application. The scheduling plane consumes this knowledge at two levels: it reorders prompts to enlarge prefix reuse, and it places KV segments across device and host with a heterogeneity-aware eviction policy to ensure that reusable KV-cache segments reside in the correct tier.

4.1 Discovering Heterogeneity

IDONTKNOWITSNAME first lifts each received prompt back to its call-site metadata. It then analyzes how each binding evolves across the request history along two dimensions, intra-call-site evolution and cross-call-site sharing, and assigns every binding a lifetime and sharing class. Separately, IDONTKNOWITSNAME learns the application's call structure as a time-annotated probabilistic finite-state machine (TPFM). The result is a per-request record that annotates each binding with its class and future reuse estimator; the scheduling plane then consumes this knowledge.

4.1.1 Lifting prompt back into metadata. The lowering process is lost when prompts cross the boundary between the runtime and the serving engine. However, because prompts are lowered by the runtime through a deterministic process, these generated requests still retain recoverable structure. The runtime emits a fixed skeleton from the declaration, consisting of the header with its signature, schema rows, and output-format tail, and drops each bound argument into a slot of that skeleton, labelled by the parameter name and placed in declaration order. The skeleton then stays static and only the binding values change.

Lifting is therefore template application run backwards. We represent a call site x by a prompt template $T_x = \langle S_x, (g_1, \dots, g_m), \tau_x \rangle$, where S_x is the static system prompt, τ_x is a request-only tail, and each segment is either a static byte string or a named slot:

$$g_i ::= \text{STATIC}(b) \mid \text{SLOT}(n, \ell),$$

where n is the parameter name and ℓ is its literal label. Rendering replaces every slot with its binding value, while splitting matches the static segments and labels to recover the bindings. A valid template must satisfy

$$\text{RENDER}(T_x, \text{SPLIT}(T_x, u)) = u$$

for every observed request u of the site.

The compiler may provide T_x directly; otherwise, the serving engine deduces it from call history. Requests are first grouped by a call-site fingerprint consisting of the model, system prompt, and call-message header. Given at least two distinct messages from one site, the engine computes their longest common subsequence of lines. Common lines form the static skeleton and the regions between them are candidate slots. Within each region, repeated labels identify parameter names and split the region into named slots. If no reliable label pattern exists, the complete region remains one conservative slot. A suffix found only in the live request is recorded separately as the request-only tail. The metadata for a programmatically generated prompt typically includes binding names and values, the system prompt, and invariant headers, as shown in Figure 1. Such metadata is obtained by applying the template to the received prompt.

Algorithm 1 Deducing a prompt template

Require: call history H_x

Ensure: T_x or $\perp$

```

1:  $(U, \tau) \leftarrow \text{NORMALIZE}(H_x)$   $\triangleright$  extract calls and strip
   request-only tail for distinct messages
2:  $K \leftarrow \text{LCS}(\{\text{LINES}(u) : u \in U\})$   $\triangleright$  find recurring lines
3:  $R \leftarrow \text{GAPS}(\text{ALIGN}(K, U))$   $\triangleright$  collect changing regions
4:  $G \leftarrow \text{SEGMENT}(K, R)$   $\triangleright$  form static segments and slots
5:  $T \leftarrow \langle \text{SYSTEM}(x), G, \tau \rangle$   $\triangleright$  assemble the template
6: if  $\text{ROUNDTrip}(T, U)$  then  $\triangleright$  verify exact reconstruction
7:   return  $T$ 
8: end if
9: return  $\perp$ 
```

4.1.2 Identifying heterogeneous evolution patterns.

Heterogeneity identification involves intra-callsite evolution patterns and cross-callsite sharing. Some components may remain static across invocations, some are shared across call sites, some accumulate as execution progresses, and some are simply transient. The identification is derived from history analysis.

Intra-callsite evolution. For each binding, we compare its value across consecutive calls of the same callsite within a session. We count and record the transitions as *same*, *extend*, and *fresh*. Concretely, *same* means the binding value remains identical, *extend* means the new value extends the old value by appending content to its end, and *fresh* denotes a distinct value.

Cross-callsite sharing. Cross-callsite sharing means a binding's value is used by multiple callsites. We check whether another callsite in the same session already carried the same value, or a prefix that the current value extends. Sharing exposes an extra opportunity to make a prefix reusable across callsites.

We characterize each binding by one of four evolution behaviors:

- *Constant*: the same literal recurs across sessions;
- *Copied*: the binding retains or reuses a value observed earlier in the same session;
- *Extended*: the new value extends an earlier value by appending;
- *Volatile*: the binding is usually replaced by a fresh value.

For each binding name n at callsite x , we record both its value origin and its evolution between consecutive invocations of x . An origin rule is considered dominant after it has been observed at least twice and accounts for at least half of the binding's observations.

A binding is *Constant* when its dominant origin is a recurring literal. Otherwise, when same-callsite transitions exist,

let

$$\rho(x, n) = \frac{N_{\text{fresh}}}{N_{\text{same}} + N_{\text{extend}} + N_{\text{fresh}}}.$$

Conceptually, the binding is *Volatile* when $\rho(x, n) > \frac{1}{2}$; otherwise, its non-fresh behavior is *Copied* or *Extended*, according to whether N_{same} or N_{extend} is larger.

A callsite invoked only once per session provides no same-callsite transitions. In that case, its dominant origin determines whether the binding is copied or extended; an unresolved or other origin is treated as volatile for stability ordering.

Independently, a binding is marked *cross-stable* when:

1. at least one observation is shared from a different callsite;
2. cross-callsite sharing accounts for at least half of its observations; and
3. either no same-callsite transitions exist or $\rho(x, n) \leq \frac{1}{2}$.

We represent the ordering stability of binding n at callsite x by

$$\sigma(x, n) = (c(x, n), \rho(x, n), r(x, n)),$$

where

$$c(x, n) = \begin{cases} 0, & \text{if } (x, n) \text{ is cross-stable,} \\ 1, & \text{otherwise.} \end{cases}$$

For a constant binding, we set

$$(\rho(x, n), r(x, n)) = (-1, 0).$$

When transition observations exist, $\rho(x, n)$ is the fresh-transition fraction, and $r(x, n)$ is the rank of the most frequent non-fresh transition:

$$r(\text{same}) = 1, \quad r(\text{extend}) = 2,$$

with $r = 3$ when no non-fresh transition has been observed.

When no transition observations exist, we set $\rho(x, n) = 1$ and derive $r(x, n)$ from the dominant origin:

$$r(\text{copy}) = 1, r(\text{extend}) = 2, r(\text{other or unresolved}) = 3.$$

Bindings are ordered lexicographically by σ , so cross-stable bindings precede other bindings, followed by increasingly volatile bindings.

4.1.3 Deriving application-level agent TPFM. IDONTKNOWITSNAME treats agents as time-annotated probabilistic finite-state machines (TPFMs). We formulate the TPFM for agent applications as follows:

$$\mathcal{A} = (Q, P, q_0, F, \mathcal{T}),$$

where:

1. Q is the set of states. Each state $q \in Q$ is labelled by an LLM callsite;
2. P denotes the transition probabilities between states.
3. $q_0 \in Q$ is the entrance callsite.
4. F specifies the termination probability of the agent at each state.

5. $\mathcal{T} = \{D_q^{\text{exec}}, D_{q,q'}^{\text{gap}}\}$ denotes the timing annotations associated with states and transitions, where D_q^{exec} is the execution-time distribution of the LLM callsite represented by state q , and $D_{q,q'}^{\text{gap}}$ is the inter-call gap distribution between the completion of callsite q and the invocation of its successor q' .

During one session, the server observes only a sequence of LLM calls. IDONTKNOWITSNAME aggregates the completed sequences into a prefix-tree acceptor whose nodes and edges retain their observed frequencies. For each application, the automaton is rebuilt whenever the number of completed sessions reaches $8 \cdot 2^k$, for $k = 0, 1, 2, \dots$. During rebuilding, IDONTKNOWITSNAME applies an Alergia-style red-blue state-merging procedure to the prefix tree, merging two states when their outcome frequencies satisfy

$$\left| \frac{c_1}{n_1} - \frac{c_2}{n_2} \right| < \sqrt{\frac{1}{2} \ln \frac{2}{\alpha} \left(\frac{1}{\sqrt{n_1}} + \frac{1}{\sqrt{n_2}} \right)}, \quad \alpha = 0.05.$$

Here, n_i is the number of observed visits to state i , and c_i is the number of those visits that produce the particular outcome. Once two states are merged, their visit, termination, transition, and timing observations are combined to construct the resulting TPFM.

4.2 Heterogeneity-aware KV Cache scheduling

The discovery plane identifies where reuse exists: the ordering-stability tuple characterizes the lifetime and sharing scope of each binding, the value-flow rules identify which future binding values can be reconstructed, and the TPFM supplies call-transition and timing observations. The scheduling plane turns this information into cache actions by exposing reusable content as a prefix and placing KV cache effectively. IDONTKNOWITSNAME combines prompt re-layout, future KV-demand estimation, deadline-driven promotion, and prediction-aware eviction to achieve this goal.

4.2.1 Exposing reuse through prompt re-layout. KV-cache reuse requires requests to share an identical token prefix. However, developer-defined declaration order may not account for this requirement, resulting in reduced prefix reuse. IDONTKNOWITSNAME therefore revises the binding order of each callsite according to the heterogeneity learned by the discovery plane.

For every binding n of callsite x , IDONTKNOWITSNAME uses the ordering-stability tuple $\sigma(x, n) = (c(x, n), \rho(x, n), r(x, n))$ defined above. Once every binding at the callsite has accumulated enough observations, the bindings are stably sorted in lexicographic order of σ and the resulting layout is frozen. Consequently, cross-stable bindings lead the request, constants precede session-varying values, bindings with a smaller fresh-transition ratio precede more volatile bindings, and same-valued bindings precede append-only bindings

Algorithm 2 Heterogeneity-aware prompt re-layout

Require: Callsite x , system prompt S_x , header H_x , bindings B , request-only tail τ_x

Ensure: Re-laid-out prompt

- 1: π_x : layout of callsite x
- 2: **if** π_x is not frozen **then**
- 3: **if** any binding has insufficient observations **then**
- 4: **return** $S_x \parallel H_x \parallel \text{RENDER}(B) \parallel \tau_x$
- 5: **end if**
- 6: $\pi_x \leftarrow \text{STABLESORT}(\text{NAMES}(B), \text{key} = \sigma(x, \cdot))$
- 7: Freeze π_x
- 8: **end if**
- 9: $V \leftarrow \text{RENDER}(B, \pi_x)$
- 10: **if** π_x is nonempty and $c(x, \pi_x(1)) = 0$ **then**
- 11: **return** $S_x \parallel V \parallel H_x \parallel \tau_x$
- 12: **else**
- 13: **return** $S_x \parallel H_x \parallel V \parallel \tau_x$
- 14: **end if**

when their preceding components tie. Bindings with identical ordering-stability tuples retain their original relative order.

Ordinarily, the re-laid-out request has the logical form

$$S_x \parallel H_x \parallel b_{\pi(1)} \parallel \dots \parallel b_{\pi(m)} \parallel \tau_x,$$

where S_x is the system prompt, H_x is the callsite-specific header and schema, π is the learned binding order, and τ_x is the request-only tail. If the first binding is cross-stable, H_x would prevent two different callsites from sharing that binding as a prefix. In this case, IDONTKNOWITSNAME places the callsite-specific header after the binding values:

$$S_x \parallel b_{\pi(1)} \parallel \dots \parallel b_{\pi(m)} \parallel H_x \parallel \tau_x.$$

This header-last layout exposes reuse across callsites without changing the system prompt or the binding contents.

4.3 Prediction-guided KV Management

Re-layout exposes reusable prefixes in incoming requests, while proactive cache placement requires anticipating future reuse. IDONTKNOWITSNAME predicts when future calls will arrive and uses binding evolution patterns to reconstruct their known prefixes, guiding fine-grained KV-cache prefetching and retention. For every live session, IDONTKNOWITSNAME maintains the observed callsite history $h_t = (x_1, \dots, x_t)$ and periodically produces a forecast

$$\widehat{C}_t = \{(x, p_x, t_{50,x}, t_{90,x})\},$$

where x is a possible future callsite, p_x is its cumulative reachability within the planning horizon, and $t_{50,x}$ and $t_{90,x}$ are the estimated median and pessimistic arrival times relative to the current timing anchor.

Single-step next-call probability estimation. Because transition frequencies can be sparse during online learning, IDONTKNOWITSNAME complements the constructed TPFM with a bounded-context predictor over callsite sequences. The predictor stores all observed callsite subsequences of length at most $K = 4$ in a context trie. For a context u , let $C_u(x)$ be the number of times callsite x follows u , let $C_u(\perp)$ be the number of sessions that terminate after u , and define

$$n_u = C_u(\perp) + \sum_x C_u(x),$$

$$T_u = |\{x : C_u(x) > 0\}| + 1[C_u(\perp) > 0].$$

The confidence assigned to context u is

$$\lambda_u = \frac{n_u}{n_u + T_u}.$$

Starting from the empty context, IDONTKNOWITSNAME applies Witten–Bell-style interpolation over increasingly long observed suffixes:

$$\widehat{P}(x \mid u) = \lambda_u \frac{C_u(x)}{n_u} + (1 - \lambda_u) \widehat{P}(x \mid \text{suffix}(u)),$$

$$\widehat{P}(\perp \mid u) = \lambda_u \frac{C_u(\perp)}{n_u} + (1 - \lambda_u) \widehat{P}(\perp \mid \text{suffix}(u)).$$

A well-supported long context can dominate the prediction, whereas a sparsely observed context is interpolated with its better-supported shorter suffixes rather than being used as a deterministic prediction. The termination outcome only consumes probability mass and does not create a successor in the prediction tree.

Cumulative reachability and arrival time. A single next-call prediction is insufficient for preparing calls several steps ahead. IDONTKNOWITSNAME therefore performs a bounded best-first expansion of possible future executions. We define a frontier element as

$$(\pi, h, q, u_{50}, s^2, d),$$

where π is the partial-path probability, h is its hypothetical callsite history, q is its current TPFM state, u_{50} is the estimated median completion time of its last call, s^2 is its accumulated squared timing spread, and d is its depth. The initial frontier contains $(1, h_t, q_t, 0, 0, 0)$.

For a candidate successor x , the new path probability is $\pi' = \pi \widehat{P}(x \mid h)$. Let q' be the state reached by x , and let

$$g_{50} = Q_{0.5}(D_{q,q'}^{\text{gap}}), \quad g_{90} = Q_{0.9}(D_{q,q'}^{\text{gap}}).$$

The estimated arrival times and spread of x are

$$a_{50} = u_{50} + g_{50}, \quad s_a^2 = s^2 + (g_{90} - g_{50})^2, \quad a_{90} = a_{50} + \sqrt{s_a^2}.$$

To expand the path beyond x , IDONTKNOWITSNAME adds the execution time of x . With

$$e_{50} = Q_{0.5}(D_{q'}^{\text{exec}}), \quad e_{90} = Q_{0.9}(D_{q'}^{\text{exec}}),$$

the successor frontier element becomes

$$(\pi', h \parallel x, q', a_{50} + e_{50}, s_a^2 + (e_{90} - e_{50})^2, d + 1).$$

The search expands the highest-probability frontier first and retains at most the three most likely successors of each frontier element. It prunes a path when its probability falls below $p_{\min} = 0.02$, median arrival exceeds the 120-second planning horizon, depth exceeds eight, or the search has expanded 64 frontier elements. When multiple paths reach the same callsite x , their path probabilities are accumulated and capped at one,

$$p_x = \min \left(1, \sum_{\varrho: \varrho \leadsto x} P(\varrho) \right),$$

while the earliest predicted arrival pair is retained for scheduling that callsite.

4.3.1 Prediction-guided KV Management. IDONTKNOWITSNAME performs proactive KV cache scheduling guided by heterogeneity and prediction. KV cache management involves deadline-driven KV promotion and prediction-aware eviction.

Deadline-driven KV promotion. Deadline-driven KV promotion anticipates the incoming prompt before it arrives, using the value-flow rules obtained in Section 4.1. Constants are inserted directly; copied values and values taken from earlier bindings and responses are resolved from the current session; and an extended value contributes the known prefix of its accumulated representation. This reconstruction proceeds binding by binding and stops at the first unresolved binding. For every predicted call, IDONTKNOWITSNAME chooses the longest cache target and uses the statically known callsite prefix as a conservative fallback.

After selecting the promotion target, IDONTKNOWITSNAME assigns it an estimated deadline. For a predicted call $c = (x, p_x, t_{50,x}, t_{90,x})$ and timing anchor a , IDONTKNOWITSNAME assigns the deadline

$$d_c = \max(t_{\text{now}}, a + t_{50,x} - \kappa(t_{90,x} - t_{50,x})), \quad \kappa = 0.5.$$

The term $\kappa(t_{90} - t_{50})$ moves uncertain jobs earlier and lets the scheduler prepare the prefix more conservatively.

The absolute pessimistic arrival time $\bar{t}_c = \max(d_c, a + t_{90})$ is used for garbage collection. If the predicted call has not arrived after $\bar{t}_c$, its cache-plan entry is considered stale and discarded.

Let U_c be the number of target-prefix tokens absent from both cache tiers, let H_c be the number resident in host memory but absent from device memory, and let R_{prefill} and R_{promote} be the profiled prefill-under-load and host-to-device transfer rates. The latest safe preparation time is estimated as

$$r_c = d_c - \frac{U_c}{R_{\text{prefill}}} - \frac{H_c}{R_{\text{promote}}}.$$

A promotion job becomes eligible whenever the engine is idle or the current time approaches r_c . Among eligible jobs, the scheduler selects the earliest deadline. During promotion, active KV cache and ongoing tasks are protected to ensure

that speculative preparation does not interfere with serving requests.

Prediction-aware eviction. IDONTKNOWITSNAME exploits both predicted usage and prompt heterogeneity in KV eviction scheduling. Device-resident radix nodes are divided into four lifecycle-aware priority bands, listed in order of eviction priority:

1. *retired*: private tail nodes belonging to completed sessions;
2. *transient*: non-reconstructible prompt tails and generated descendants that are unlikely to be reused;
3. *unclassified*: ordinary cache not covered by the current plan;
4. *planned*: nodes required by one or more active predictions.

The scheduler uses LRU order within the *retired*, *transient*, and *unclassified* bands. Nodes in the *planned* band are sorted by aggregated protection score and then by recency.

Band transitions occur at three events. After a request completes, nodes beyond its longest pre-arrival reconstructible prefix are marked *transient*. When the session terminates, its *transient* tails become *retired*. At every plan refresh, IDONTKNOWITSNAME removes the previous planned protections before applying the new scores, preventing stale predictions from retaining priority.

The host tier then serves as a recovery layer. It evicts retired entries first and otherwise follows LRU. A prediction error is therefore less likely to turn into a full recomputation.

Additionally, IDONTKNOWITSNAME converts every live prediction into an eviction-protection score:

$$S_c(t) = p_x \exp \left(-\frac{\max(0, d_c - t)}{\tau} \right), \quad \tau = 60 \text{ s.}$$

This score is added to the eviction priority of every radix-tree node on the prefix's path. Nodes in the *planned* band are evicted in increasing order of accumulated score, then by LRU.

5 Evaluation

5.1 Experimental Setup

Configurations. We evaluate IDONTKNOWITSNAME on a single RTX-3090 with Qwen-8B (bf16). IDONTKNOWITSNAME is implemented on SGLang with HiCache and chunked prefill [1] enabled. For baseline comparison, we select vanilla SGLang and two opaque-request serving systems, CacheScout and Continuum [12, 17].

Workloads. We evaluate IDONTKNOWITSNAME using two representative agent applications: a fact-checking agent and a coding agent. Using these applications, we execute two benchmarks: HoVer [9], a multi-hop fact verification benchmark requiring evidence collection and reasoning across

Algorithm 3 Prediction-aware eviction

```

Require: Cache tier  $C$ , required free space  $M$ , live predic-
tions  $\mathcal{P}$ , current time  $t$ 
Require: Lifecycle labels updated on request completion
and session termination
Ensure: Whether at least  $M$  free space is available
1: while  $\text{FREESPACE}(C) < M$  do
2:    $E \leftarrow \text{EVICTABLELEAVES}(C)$   $\triangleright$  Exclude locked nodes
3:   if  $E = \emptyset$  then
4:     return false
5:   end if
6:   for all  $v \in E$  do
7:      $s_v \leftarrow 0$ 
8:     if  $v$  is retired then
9:        $b_v \leftarrow 0$ 
10:    else if  $v$  is transient then
11:       $b_v \leftarrow 1$ 
12:    else
13:       $b_v \leftarrow 2$   $\triangleright$  Unclassified
14:    end if
15:    if  $C$  is the device tier then
16:       $\mathcal{P}_v \leftarrow \{c \in \mathcal{P} : v \in \text{PATH}(c)\}$ 
17:      if  $\mathcal{P}_v \neq \emptyset$  then
18:         $b_v \leftarrow 3$   $\triangleright$  Planned
19:         $s_v \leftarrow \sum_{c \in \mathcal{P}_v} S_c(t)$ 
20:      end if
21:    else
22:       $b_v \leftarrow \min(b_v, 1)$   $\triangleright$  Host: retired first, then
LRU
23:    end if
24:     $K_v \leftarrow (b_v, s_v, \text{LASTACCESS}(v))$ 
25:  end for
26:   $v^* \leftarrow \arg \min_{v \in E} K_v$   $\triangleright$  Lexicographic order
27:   $\text{EVICT}(C, v^*)$ 
28: end while
29: return true

```

multiple Wikipedia articles, and HumanEval [2], a code-generation benchmark that evaluates the functional correctness of generated programs using unit tests.

5.2 Task quality and execution scale

IDONTKNOWITSNAME introduces prompt re-layout, which changes the ordering inside the prompt. Such a transformation should preserve comparable task quality and scale. As shown in Figure 3 and Table 1, all configurations achieve comparable task accuracy. At the same time, output tokens per session remain essentially the same. The observed serving improvements do not come at the cost of output quality or a reduction in task scale.

Table 1. Application-level task quality across serving configurations.

Method	Fact Checking	Code Generation
SGLang (LRU)	57.2%	47.6%
CacheScout	57.2%	47.2%
Planner only	54.9%	46.2%
Re-layout only	56.4%	45.5%
Ours	58.0%	45.3%

5.3 Evaluation Analysis

Our evaluations on IDONTKNOWITSNAME address the following research questions:

1. How much does IDONTKNOWITSNAME improve end-to-end serving performance?
2. How much do the two scheduling mechanisms of IDONTKNOWITSNAME contribute to the improvement?
3. How do the two mechanisms of heterogeneity-aware KV-cache scheduling in IDONTKNOWITSNAME interact and reinforce each other?

5.3.1 RQ1: How IDONTKNOWITSNAME improves end-to-end performance. We evaluate IDONTKNOWITSNAME on two representative agent applications: a fact-checking agent on the HoVer dataset and a coding agent on the HumanEval dataset. We use mean job completion time (JCT) and mean time to first token (TTFT) as the end-to-end performance metrics.

JCT. As shown in Figures 5a, 5b, 4b, and 4a, IDONTKNOWITSNAME consistently reduces mean JCT over the vanilla SGLang baseline, achieving $1.3\times$ – $1.7\times$ speedup on the fact-check agent and $1.1\times$ – $1.3\times$ on the coding agent. On the fact-check agent, Continuum performs comparably to SGLang at low concurrency but its advantage vanishes as load increases, whereas IDONTKNOWITSNAME sustains its gains across all concurrency levels. On the coding agent, IDONTKNOWITSNAME outperforms every other system; CacheScout and Continuum degrade sharply at high concurrency and fall behind SGLang.

TTFT. IDONTKNOWITSNAME also lowers mean TTFT across all workload configurations (Figures 5c and 5d). On the fact-check agent, the reduction is substantial at every concurrency level, while Continuum and CacheScout offer little or no improvement over SGLang. On the coding agent, IDONTKNOWITSNAME matches or beats all baselines up to a concurrency of 40; at the highest concurrency, KV-only achieves a slightly lower TTFT.

Multi-Program serving of IDONTKNOWITSNAME. To understand how IDONTKNOWITSNAME performs when serving multiple programs, we evaluate IDONTKNOWITSNAME with a mixed workload of the fact-checking and coding agents

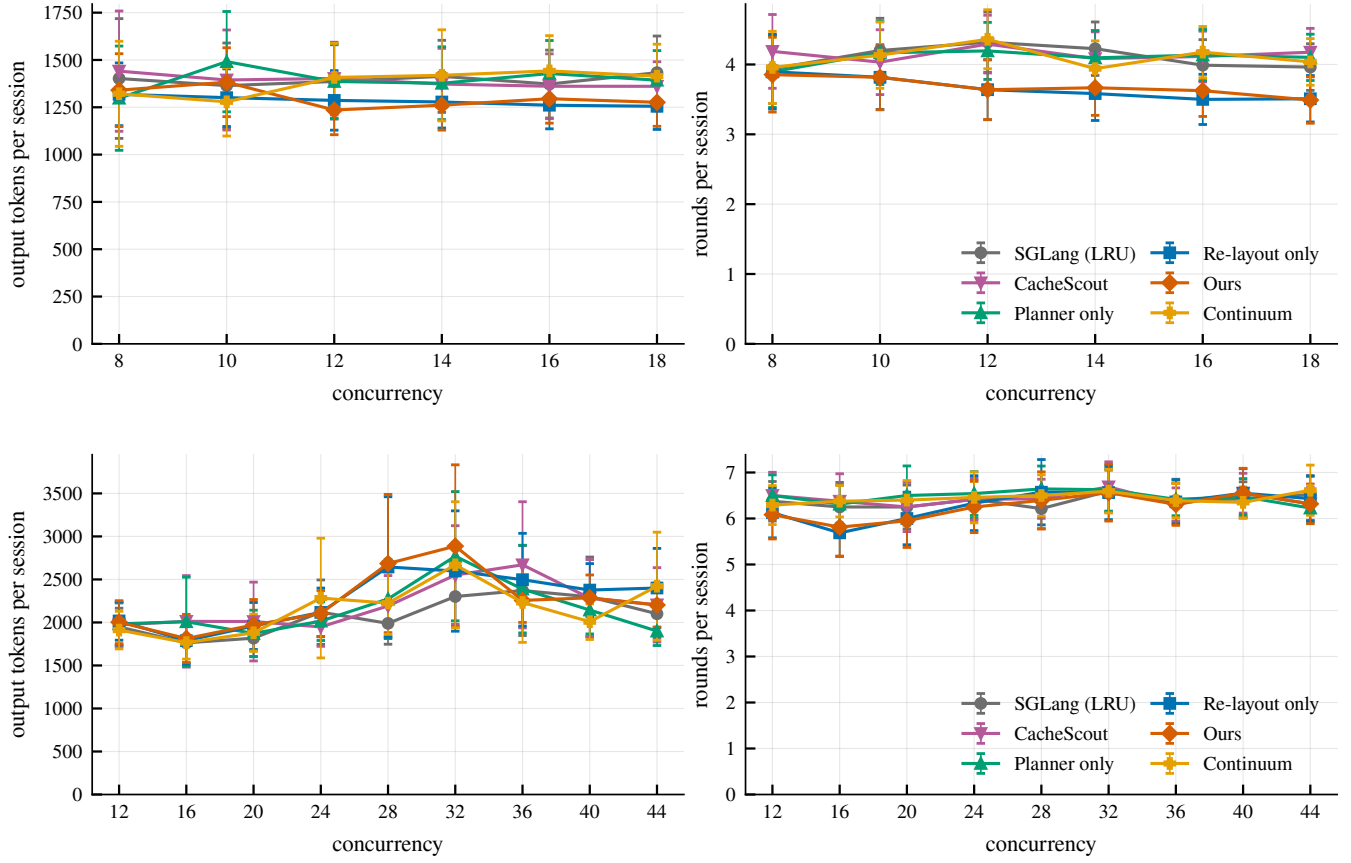


Figure 3. Application-level task quality under varying concurrency for the HoVer fact-checking agent (left) and the HumanEval coding agent (right).

sharing one server. We set concurrency at 16: 4 lanes consecutively launch fact-checking sessions and 12 lanes consecutively launch coding sessions, and all 16 lanes run in parallel. We select this 1:3 ratio based on the KV-cache footprint of the two agents. Every system first observes two sessions of each program, which are excluded from the numbers. We report throughput and mean TTFT for this evaluation. Throughput is calculated as the number of sessions completed per minute while all 16 lanes are busy.

Table 2 shows the results. The throughput of IDONTKNOWITSNAME has a $1.51\times$ speedup over SGLang, with a mean TTFT speedup of $2.37\times$. Meanwhile, CacheScout and Continuum barely improve throughput over SGLang, with a $1.02\times$ speedup; their mean TTFT shows up to a $1.33\times$ speedup over SGLang. The ablation arms show the same pattern as in the single-program experiments. Re-layout alone accounts for most of the miss-rate reduction (58.3%) and a $1.36\times$ throughput gain. The planner alone, which stays inert on a single coding workload, now delivers a $1.23\times$ gain with 5.5 points

fewer misses. In a mixed pool, the eviction decision is between programs with different reuse patterns, which is exactly the decision the learned per-program model informs.

The mix also tests whether the structure IDONTKNOWITSNAME learns is per program. Under interleaving, IDONTKNOWITSNAME’s next-call prediction is correct for 89% of the fact-checking calls and 80% of the coding calls.

5.3.2 RQ2: How much the two scheduling mechanisms of IDONTKNOWITSNAME contribute to the improvement.

The improvement of IDONTKNOWITSNAME comes from more efficient KV cache scheduling. From Figure 6, it can be clearly observed that the performance gain comes from a lower KV cache miss rate. IDONTKNOWITSNAME’s heterogeneity-aware KV cache scheduling helps keep useful KV entries in cache. This scheduling mechanism incorporates two aspects, heterogeneity-aware prompt re-layout and prediction-guided KV management. We thus investigate detailed serving status to understand how the two KV scheduling mechanisms contribute to the performance improvement.

Table 2. Performance of IDONTKNOWITSNAME and baselines on a mixed workload of fact-checking and coding agents. IDONTKNOWITSNAME improves throughput by 1.51 $\times$ and reduces mean TTFT by 58% over SGLang, with the KV-cache miss rate falling from 87.0% to 56.7%.

Method	Throughput (sess/min)	Speedup	Mean TTFT (ms)	KV-cache status		
				device	host	miss
SGLang (LRU)	3.64	1.00 $\times$	6266	10.5%	2.5%	87.0%
CacheScout	3.70	1.02 $\times$	4712	14.1%	2.3%	83.6%
Continuum	3.72	1.02 $\times$	4964	10.1%	4.5%	85.4%
Planner only	4.47	1.23 $\times$	4254	13.9%	4.6%	81.5%
Re-layout only	4.95	1.36 $\times$	4164	19.2%	22.6%	58.3%
Ours	5.51	1.51$\times$	2642	31.3%	12.0%	56.7%

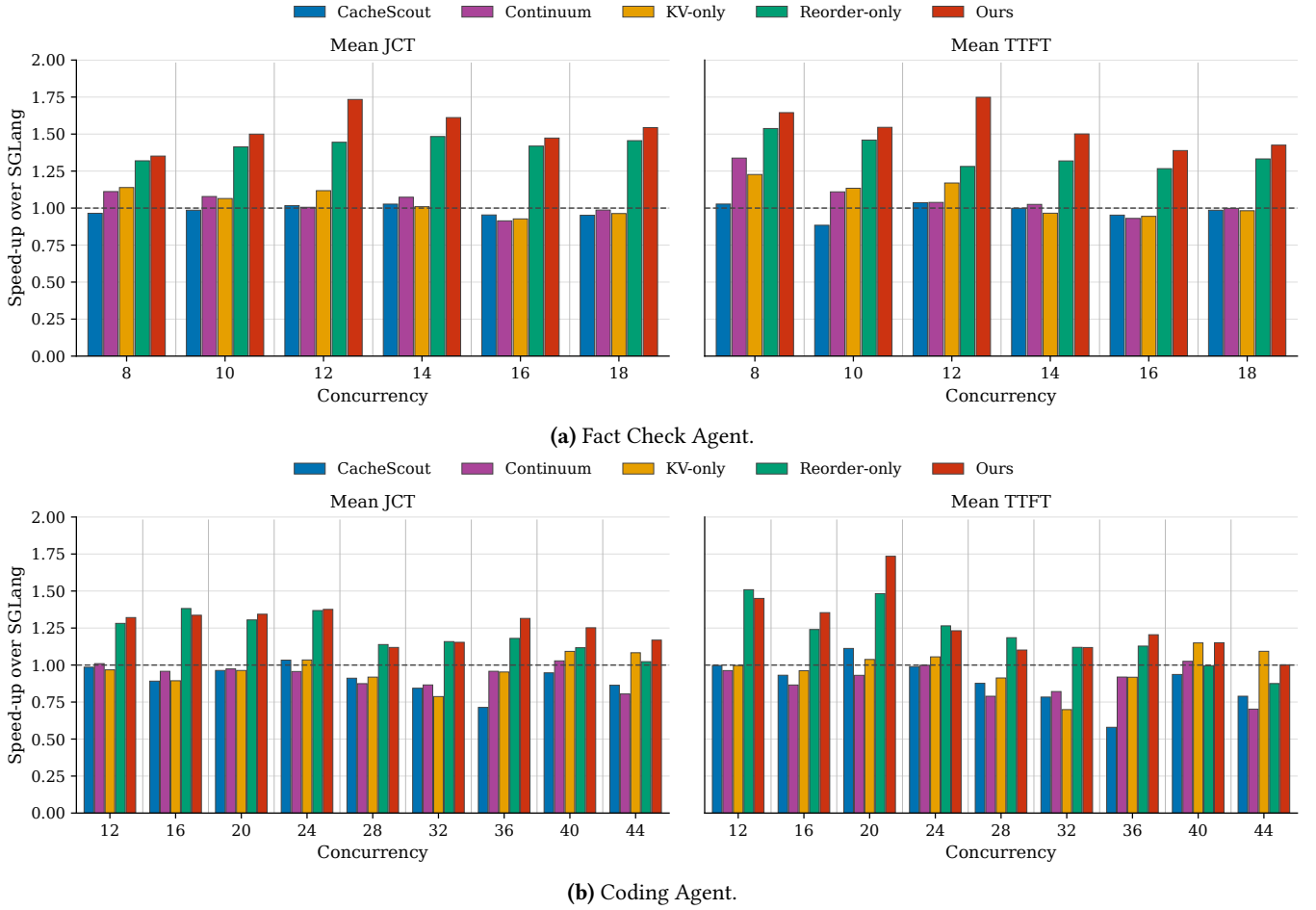


Figure 4. Speed-up over SGLang in mean JCT and mean TTFT across concurrency levels.

Speed-up introduced by heterogeneity-aware prompt re-layout. To investigate the effects of prompt re-layout, we conduct an ablation experiment that disables IDONTKNOWITSNAME’s KV scheduling mechanism and uses only heterogeneity-aware prompt re-layout, shown as the *re-layout only* arm in Figure 6. When serving the coding agent, which is decode-heavy, re-layout obtains nearly all of the

optimization when concurrency is below 28. But as the workload continues to grow, re-layout alone cannot sustain useful KV, the cache miss rate begins to drop, leading to a larger gap between it and full IDONTKNOWITSNAME. Across sessions of the fact-check agent, re-layout only keeps a consistently slightly higher cache miss rate, which causes a performance deficit compared to full IDONTKNOWITSNAME.

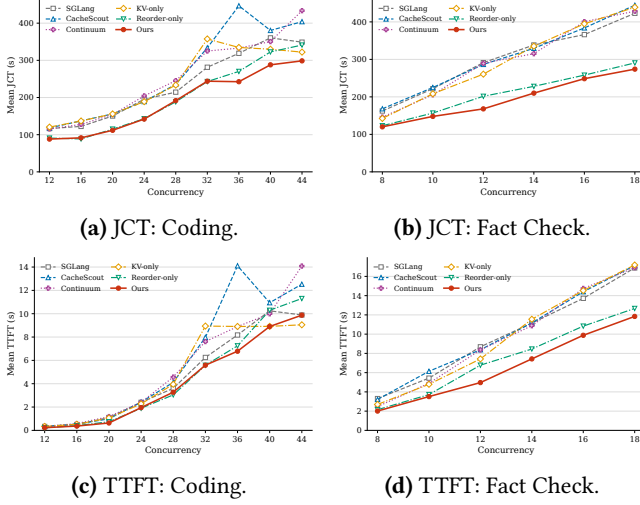


Figure 5. Mean JCT and TTFT across concurrency levels for the Coding Agent and Fact Check Agent.

These evaluation results demonstrate that heterogeneity-aware prompt re-layout by itself helps enhance performance. Across all workload pressures, heterogeneity-aware prompt re-layout improves performance by introducing more opportunities for KV sharing, thus reducing cache misses and optimizing serving performance. It can even match full IDONTKNOWITSNAME in some cases, but still falls behind full IDONTKNOWITSNAME under high workload pressure.

Effects of Prediction-guided KV Management. Adopting a similar methodology, we create an ablation arm that activates only the KV management of IDONTKNOWITSNAME, which is the KV-only arm in Figure 6. This mechanism shows substantially different behavior from prompt re-layout. In fact-check agent serving sessions, it only works under low pressure, where $c \leq 14$, which is very similar to other KV management frameworks: Continuum and CacheScout also work only under very low concurrency levels. Moreover, this arm, together with Continuum and CacheScout, stays inert when serving the coding agent.

The reason for this phenomenon is straightforward: KV cache management needs space to operate. The effect of prediction-guided KV management comes from keeping useful entries in cache and preventing them from being evicted. However, when pressure is high, ongoing requests compete for memory and no KV cache can survive. Consequently, the effect of KV cache management diminishes under high workload pressure. A similar phenomenon can be observed for Continuum and CacheScout. But the effect of IDONTKNOWITSNAME’s prediction-guided KV management disappears much later than that of Continuum and CacheScout. On the fact-check agent, Continuum and CacheScout lose their effectiveness at $c = 10$, while IDONTKNOWITSNAME’s prediction-guided KV management survives to $c = 14$.

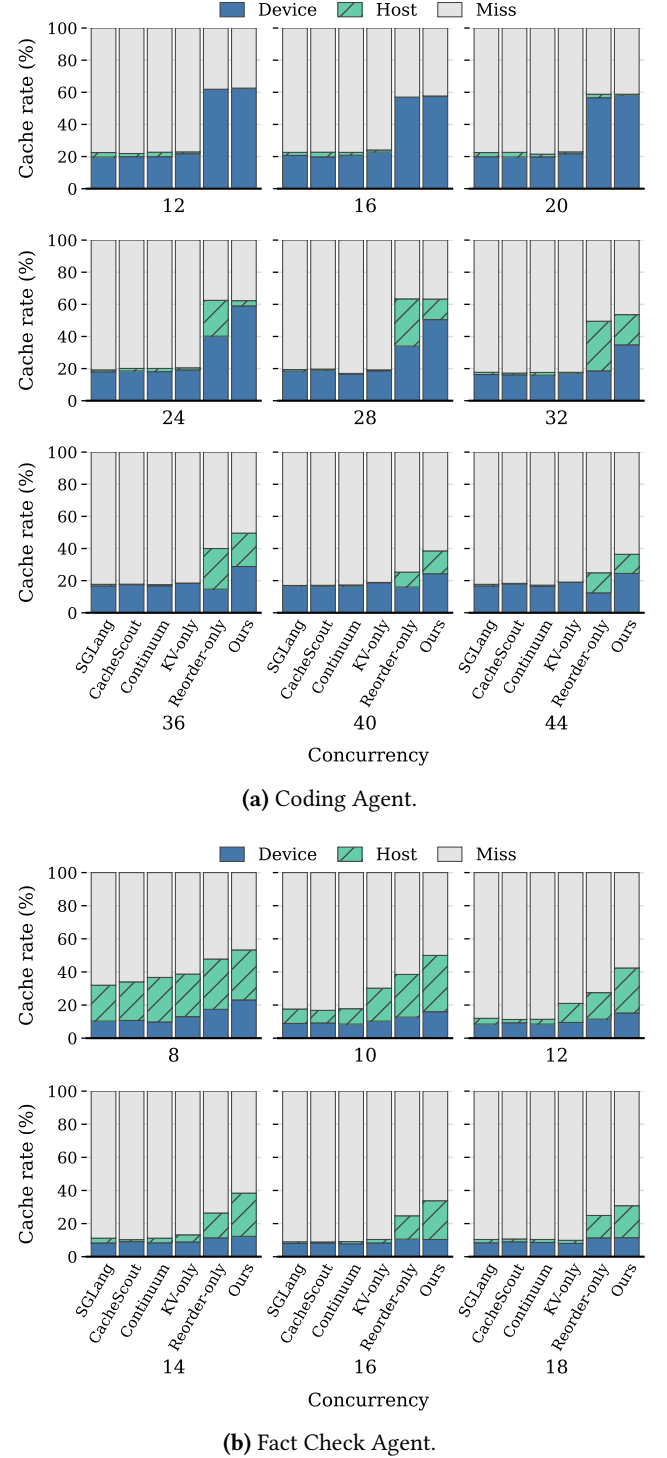


Figure 6. End-to-end KV-cache status (device hit, host hit, miss) under different concurrency levels.

5.3.3 RQ3: How the two mechanisms in IDONTKNOWITSNAME’s KV scheduling interact and reinforce each other. To analyze the interaction and mutual reinforcement

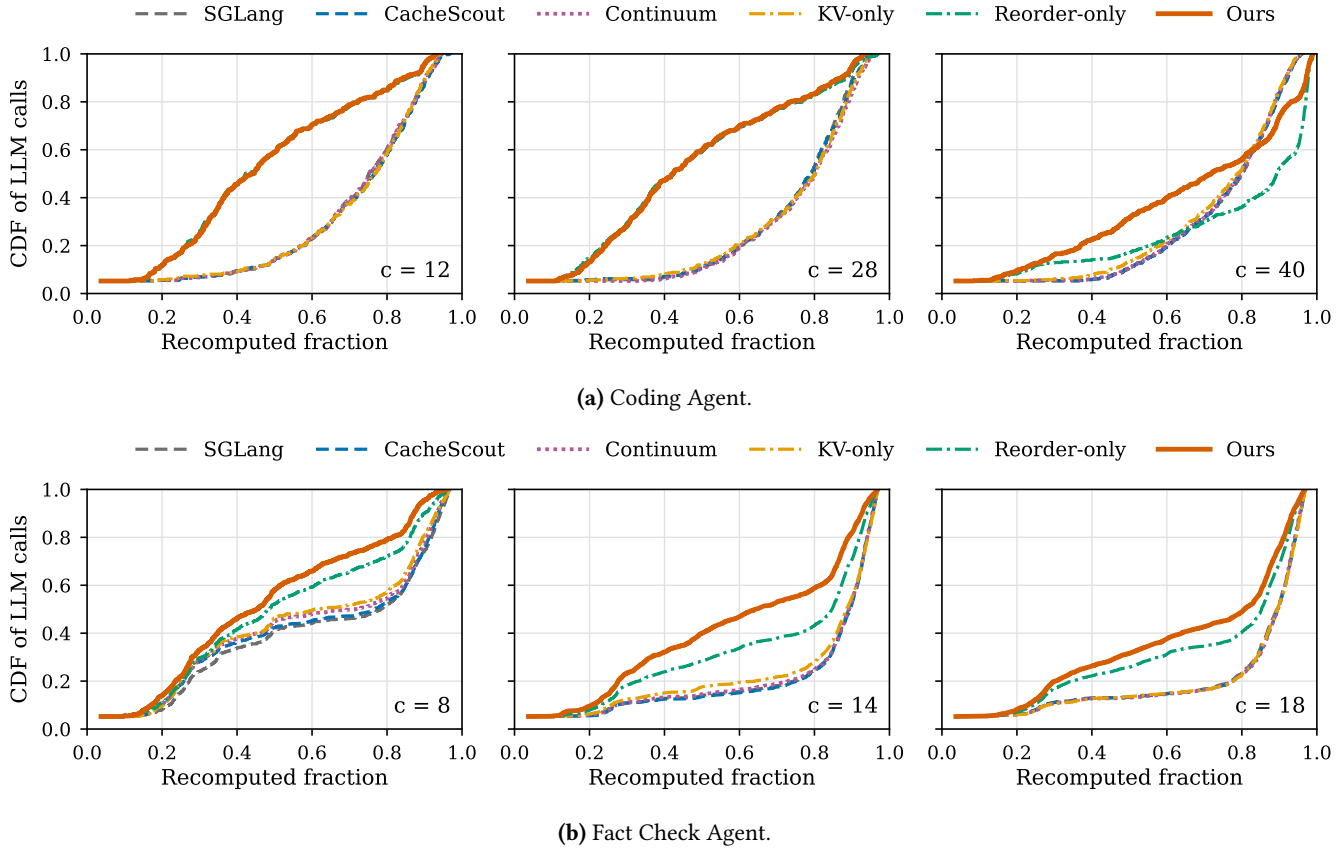


Figure 7. CDF of the recomputed-token fraction per LLM call at representative concurrency levels.

of the two mechanisms, we scrutinize KV cache statistics and investigate some representative pressure points. We first analyze the behavior of `IDONTKNOWITSNAME` under high workload.

When serving the coding agent, *re-layout only* obtains the same performance as full `IDONTKNOWITSNAME`. But under high workload pressure, full `IDONTKNOWITSNAME` clearly works better: enabling `IDONTKNOWITSNAME`'s KV cache management mechanism further improves serving performance. Comparing the cache status in Figure 6a, *re-layout only* initially keeps the same cache miss rate (when $c \leq 28$) but its device miss rate increases, and the host tier catches the mistakenly evicted KV entries. The efficient host-to-device KV cache loading in SGLang fills the gap, so performance does not diverge at those points. When concurrency grows beyond 32, full `IDONTKNOWITSNAME` begins to outperform *re-layout only*: it has fewer cache misses, leading to better performance.

This behavior also emerges on the fact-check agent. As mentioned above, `IDONTKNOWITSNAME`'s KV management algorithm only works under low pressure, where $c \leq 14$. Heterogeneity-aware prompt re-layout is effective across

all pressures because it creates more KV sharing opportunities, which leaves space for KV scheduling. Therefore, heterogeneity-aware prompt re-layout and prediction-guided KV management are not two isolated mechanisms: prompt re-layout creates space, and KV management further improves cache efficiency.

We select some representative workload pressures and investigate how many tokens are recomputed. We choose $c = 12, 28, 40$ for the coding agent and $c = 8, 14, 18$ for the fact-check agent. As shown in Figure 7a, prompt re-layout maintains exactly the same number of recomputed tokens as full `IDONTKNOWITSNAME`, but at $c = 40$ things are different: re-layout only is left behind, and adding KV scheduling further reduces the recomputed tokens. In the regime of the fact-check agent, all KV scheduling mechanisms reduce the recomputed tokens; CacheScout, Continuum, and `IDONTKNOWITSNAME` all enhance cache efficiency. Similarly, when the workload is high, `IDONTKNOWITSNAME`'s prediction-guided KV management can add optimization on top of heterogeneity-aware prompt re-layout, but KV scheduling by itself cannot take effect.

6 Related Work

LLM programming abstractions. MTP and its ByLLM implementation derive LLM operations from typed declarations and semantic information [3]. DSPy separates declarative signatures and modules from prompt construction and optimization [10]. NVIDIA Object-Oriented Agents embeds LLM behavior in Python objects [6], while SIGIL compiles skill documents into typed harnesses with explicit control flow [4]. These systems make program structure available during application construction. IDONTKNOWITSNAME studies how the resulting structure can inform serving: named bindings expose value evolution and sharing that token-prefix lookup alone does not identify. It consumes supplied templates or infers them from observed requests, and uses the recovered structure for cache management.

Memory-efficient inference and hierarchical caching. vLLM's PagedAttention improves KV allocation and sharing through paged memory management [11]. SGLang combines a structured programming interface with RadixAttention and cache-aware scheduling [19]. Sarathi-Serve uses chunked prefill to improve the throughput-latency tradeoff [1]. CachedAttention retains conversation KV in a hierarchy and overlaps data movement with computation [7]; Mooncake builds a distributed cache and scheduling architecture around KV reuse [14]. These mechanisms provide allocation, lookup, transfer, and execution support. IDONTKNOWITSNAME builds on SGLang and adds decisions about the serialized prefix and its future demand, using binding lifetimes to determine what can be reused and prepared.

Modular and non-prefix attention reuse. Prompt Cache explicitly defines reusable prompt modules through schemas and manages their positions for attention-state reuse [8]. CacheBlend targets inputs assembled from multiple text chunks, reusing cached states beyond the common prefix and selectively recomputing tokens to restore cross-chunk interactions [15]. These works recognize that a prompt contains reusable components. IDONTKNOWITSNAME instead exposes common prefixes by reorganizing labelled bindings according to their observed evolution, then uses ordinary exact prefix caching on the resulting requests. It does not transplant KV tensors computed under a different preceding context. Exact cache reuse for a transformed request does not imply that the transformation preserves model outputs; the task-quality evaluation addresses that separate question.

Workflow-aware agent serving. KVFlow represents an agent schedule as an Agent Step Graph and uses distance to future execution for node-level eviction and overlapped prefetching [13]. PBKV predicts several future agent invocations in dynamic workflows using historical executions and current context, then conservatively guides eviction and prefetching [18]. CacheScout learns agent transitions online without a predefined workflow graph [17]. Continuum

uses bounded retention through KV-cache time-to-live to preserve continuity across tool calls [12]. Pythia exposes workflow semantics through a serving interface to drive speculative cache management, lookahead scheduling, and autoscaling [16].

These systems establish the value of anticipating agent execution, including managing individual cache nodes and shared prefixes. IDONTKNOWITSNAME's distinction is the coupling of that anticipation with the structure inside generated prompts. It learns which bindings are constant, copied, extended, or volatile; uses this information to change the reusable prefix; and reconstructs the known portion of predicted requests to guide preparation and retention. Thus, workflow prediction supplies the likely destination and time of reuse, while binding analysis supplies the content and boundary of the reusable prefix.

7 Conclusion

The inherent structure and heterogeneity of programmatically generated prompts create opportunities for optimization. IDONTKNOWITSNAME identifies and exploits these opportunities rather than treating each prompt as a monolithic unit, thereby unlocking proactive optimizations that improve performance and efficiency. Our evaluation demonstrates that IDONTKNOWITSNAME's methodology successfully creates a more efficient LLM serving system.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 117–134.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [3] Jayanaka L. Dantanarayana, Yiping Kang, Kugesan Sivasothynathan, Christopher Clarke, Baichuan Li, Savini Kashmira, Krisztian Flautner, Lingjia Tang, and Jason Mars. 2025. MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 1176–1204. doi:10.1145/3763092
- [4] Jayanaka L. Dantanarayana, Savini Kashmira, Lingjia Tang, and Jason Mars. 2026. SIGIL: Compiling Agent Skills into Typed Harnesses. arXiv:2607.27309 [cs.SE] <https://arxiv.org/abs/2607.27309>

- [5] DSPy Contributors. 2026. ChatAdapter: DSPy API Reference. Online documentation. <https://dspace.ai/api/adapters/ChatAdapter/> Accessed September 14, 2026.
- [6] Paul Furgale, Severin Klingler, James Nolan, Matt Staats, Gaia Di Lorenzo, Elisa Martinez Abad, Christian Schüller, Razvan Dinu, Alessio Devoto, Pascal Berard, Gal Kaplun, Elad Sarafian, Riccardo Roveri, Leon Derczynski, and Ricardo Silveira Cabral. 2026. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents. arXiv:2607.20709 [cs.AI] <https://arxiv.org/abs/2607.20709>
- [7] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 111–126. <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [8] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, Vol. 6. 325–338. https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html
- [9] Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. 2020. HoVer: A Dataset for Many-Hop Fact Extraction And Claim Verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 3441–3460.
- [10] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [12] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230 [cs.OS]
- [13] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. arXiv:2507.07400 [cs.DC] <https://arxiv.org/abs/2507.07400>
- [14] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [15] Jiayi Yao, Hanchen Li, Yuhua Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 16 pages. doi:10.1145/3689031.3696098
- [16] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. 2026. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. arXiv:2604.25899 [cs.MA] <https://arxiv.org/abs/2604.25899>
- [17] Rui Zhang, Chaeun Kim, Shaoting Feng, Kuntai Du, Yuhua Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. 2026. Learning Agent Execution for KV-Cache Management in Agentic Serving. arXiv:2608.14624 [cs.AI]
- [18] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management. arXiv:2605.06472 [cs.LG] <https://arxiv.org/abs/2605.06472>
- [19] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37.